La realidad de Java en la industria

1. EDITOR

Main.java

Escribes código Java legible para humanos en cualquier editor (VS Code, IntelliJ, etc.).

Java Compiler Online

2. COMPILER (javac)

Main.class

javac traduce tu .java a bytecode: instrucciones neutrales al SO que ningún procesador entiende directamente.

3. JVM

nativo

La Java Virtual Machine interpreta el bytecode a código máquina del SO. Write once, run anywhere.

Write Once, Run Anywhere — el bytecode es portátil: la JVM existe para Windows, macOS, Linux…

Variables, Constantes y Funciones

VARIABLES / CONSTANTES

Contenedor que guarda un valor en memoria.

```
public double interes = 2.5; // variable
public final double PI = 3.14159; // constante
[acceso] [final] [tipo] [nombre] = [valor];
```

FUNCIÓNES / MÉTODOS

Bloque de código reutilizable que puede recibir datos y retornar un resultado.

```
public final int sumar(int a, int b) {
  return a + b;
}

[acceso] [final] [tipo] [nombre] (parámetros) {}
```

Tipos Primitivos — Valores directos en memoria

Java tiene 8 tipos primitivos. A diferencia de los objetos, se almacenan directamente en el stack.

Tipo	Tamaño	Rango / Valores	Uso típico
`boolean`	1 bit	true / false	Condiciones lógicas
`byte`	8 bits	-128 → 127	Datos binarios
`short`	16 bits	-32,768 → 32,767	Números pequeños
`int`	32 bits	-2.1B → 2.1B	El entero por defecto
`long (L)`	64 bits	-9.2×10^18 → 9.2×10^18	Números muy grandes
`float (f)`	32 bits	~6-7 decimales significativos	Decimales (menos exacto)
`double`	64 bits	~15-16 decimales significativos	El decimal por defecto
`char`	16 bits	\u0000 → \uFFFF	Un carácter Unicode

Operadores Binarios y Tablas de Verdad

OPERADORES LÓGICOS

&&	**AND** true si AMBOS son true
||	**OR** true si AL MENOS UNO es true
!	**NOT** invierte el valor booleano
^	**XOR** true si los valores son DISTINTOS

COMPARADORES RELACIONALES

==	igual	!=	distinto
<	menor	>	mayor
<=	menor-igual	>=	mayor-igual

TABLAS DE VERDAD

A	B	A && B	A || B	A ^ B
true	true	true	true	false
true	false	false	true	true
false	true	false	true	true
false	false	false	false	false

Tabla NOT (!)

A	!A
true	false
false	true

Estructuras de control: Anatomía de los condicionales (if / else)

IF: Evalúa una condición booleana. Si es true, ejecuta el bloque asociado.

```
if (condición) {
    x = 0;
    y = 1;
}

if (condición) x = 0;
```

ELSE: Define qué se hace si la condición no se cumple

```
if (condición) {
    // true
} else {
    // false
}
( condición ) ?  true  :  false;
```

COMPLEJIDAD CICLOMÁTICA

```
if (x != 0) {
    if (x > 0){
        // x es positivo
    } else {
        // x es negativo
    }
} else {
    // x es 0
}
```

```
if (x == 0) {
    // x es 0
}
if (x > 0) {
    // x es positivo
}
if (x < 0) {
    // x es negativo
}
```

Estructuras de control: Anatomía del switch

Switch clásico

```
switch (dia) {
  case 1:
    // el dia es lunes
    break;
  case 2:
    // el dia es martes
    break;
  case … : // etc
  default:
    // error
}
```

Switch moderno

```
switch (dia) {
  case 1 -> // el dia es lunes ;
  case 2 -> // el dia es martes ;
  case 3 -> // el dia es miercoles ;
  case 4 -> // el dia es jueves ;
  case 5 -> // el dia es viernes ;
  case 6 -> // el dia es sabado ;
  case 7 -> // el dia es domingo  ;
  default -> // error ;
}
```

Estructuras de control: Bucles

BÁSICOS

```
while (condición y/o actualización) {
    ...
}
```

```
do {
    ...
} while (condición y/o actualización);
```

```
for  ( init ; condición ; actualización )  {
    ...
}
```

ITERADOR

```
for  ( Tipo elemento : iterable )  {
    ...
}
```

RECURSIVIDAD

```
public int bucle(parámetros) {
    ...
    if (condición y/o actualización) {
        bucle(parámetros)
    }
    ...
}
```